

HavenWorld Build Guide

Part 6 — Phase 3 Additions

Legacy Features, Enhanced Gameplay Systems & Full Social World Expansion

Author: Daniel **Date:** Friday, 18 September 2026 **Location:** Anoka, MN, USA **Time:** 15:52 CDT

Series: Parts 1-6 **Phase:** Phase 3 — Additions **Status:** In Progress **Stack:** Babylon.js • Express 5 • Prisma • PostgreSQL 16 • Socket.io 4

Build Guide Series — Part Reference

Part

Phase

Coverage

Part 1

Phase 0

Mac Setup — Dev environment, GitHub, Cloudflare Pages, CI/CD pipeline

Part 2

Phase 0

Linux Oracle Cloud Setup — VM, PostgreSQL, Redis, Nginx, SSL, PM2

Part 3

Phase 1

Mac Client — Phaser 3 (deprecated), Babylon.js migration

Part 4A

Phase 1

Linux Server — Express 5, Prisma ORM, PostgreSQL schema (Users through InviteCodes)

Part 4B

Phase 1

Linux Auth, Socket.io, RoomManager, Moderation

Part 4C

Phase 1

Linux Admin, Invite Codes, Deployment, Alpha Checklist

Part 5A

Phase 2

3D Engine Transition — Babylon.js, ArcRotateCamera isometric-style, room scenes

Part 5B

Phase 2

3D Avatars, Blender Pipeline, GLB/Draco compression, Personal Rooms

Part 6 

Phase 3

Additions — Mini-games, Economy, Pets, Crafting, Social Systems (THIS DOCUMENT)

⚙️ Established Tech Stack — Phase 3 Builds On This Foundation

Layer

Technology

Detail

Frontend

Babylon.js

3D scenes, ArcRotateCamera, WebGPU/WebGL2 — hosted on Cloudflare Pages
(havenworld.pages.dev)

Backend

Node.js 22 LTS • Express 5 • Socket.io 4

REST API + real-time events

ORM & DB

Prisma ORM • PostgreSQL 16

Existing tables: Users, Avatars, Items, Inventories, Rooms, RoomFurniture,
ChatMessages, Friends, Reports, InviteCodes

Cache

Redis 7

Session store, pub/sub, TTL queues

Infra

Oracle Cloud Free Tier ARM VM

4 OCPUs, 24 GB RAM — Nginx reverse proxy • PM2 process manager

Hosting

Cloudflare Pages • nip.io DNS

havenworld.pages.dev / [oracle-ip].nip.io — zero paid services

Table of Contents

Intro Phase 3 Introduction & Design Philosophy 3

§1 Haven Park Fishing Mini-Game 4

§2 Loft Privacy Modes & Doorbell System 7

§3 Loft Guestbook & Tip Jar 10

§4 Procedural Zero-Asset Sound Engine 13

§5 Anti-Scam P2P Trading System 15

§6 Dual-Currency Economy & 48-Hour Rotating Shop 18

§7 Pet Companions & Autonomous AI 22

§8 Pizza Chef Mini-Game 26

§9 Workshop Crafting & Furniture Recycling 29

§10 Player Passport & Achievement Stamps 32

§11 Loft Ambient Moods 36

§12 Seasonal Events System 39

§13 Player Club System 42

§14 Emote Wheel & Expression System 45

§15 Daily Quest System 47

§16 Photo Booth & Room Screenshots 49

App. A Migration Consolidation — Full Phase 3 Prisma Plan 52

App. B Phase 3 Deployment Checklist 53

App. C Implementation Sprint Order & Feature Summary Table 54

Introduction — Phase 3 Design Philosophy

Phase 3 expands HavenWorld from its functional alpha established through Phases 0–2 into a rich, fully-featured social virtual world. All features in this phase layer onto the existing Babylon.js 3D client and Express/Socket.io/Prisma server without requiring architectural rewrites or introducing any paid infrastructure. Every feature is engineered for the free-tier Oracle Cloud ARM VM and Cloudflare Pages hosting established in Part 2.

This document is the authoritative reference for Phase 3 implementation. Each section follows a consistent structure: **What it is**, **enhanced design considerations** beyond the minimum viable version, **new Prisma schema additions**, **Socket.io event contracts**, and **numbered implementation steps** tagged **MAC** or **LINUX** to indicate the machine on which each step is performed.

★ Design Principle for Phase 3

No feature in Phase 3 requires a paid service, custom domain, or VM upgrade. Every system runs on the Oracle Cloud Free Tier ARM VM (4 OCPUs, 24 GB RAM) and Cloudflare Pages. Where external storage is needed (Photo Gallery), Oracle Cloud Object Storage always-free 10 GB tier is used. HavenWorld remains 100% free to operate through all of Phase 3.

🐼 [MAC] Tag Convention

Steps marked **MAC** are performed on your local macOS development machine — typically involving Babylon.js client code, TypeScript compilation, Blender assets, or Cloudflare Pages deployment.

[LINUX] Tag Convention

Steps marked **LINUX** are performed over SSH on the Oracle Cloud ARM VM — typically involving Prisma migrations, server-side TypeScript, Socket.io handlers, cron jobs, or PM2 reloads.

\$1 Haven Park Fishing Mini-Game

File references: `src/shared/fishing.ts` `src/client/fishing/FishingScene.ts`
`src/server/fishing.ts`

An interactive fishing mini-game positioned at Haven Park's designated water spot. Players cast a line and fight a tension-bar mechanic to reel in fish, earning HavenCoins on success. The game is server-authoritative to prevent client-side cheating — the client sends reel positions and the server determines the outcome.

Fish Catalog & Rarity Tiers

Species	Rarity	Coin Reward	Weight Range	Notes
Pond Guppy	Common	5 coins	0.5-1.5 lbs	Default; always present
Cozy Neon Tetra	Uncommon	12 coins	1.0-3.0 lbs	Slightly faster tension drain
Shimmering Golden Perch	Rare	35 coins	3.0-6.0 lbs	Performs 2 rage runs
Radiant Haven Koi	Legendary	150 coins + badge stamp	5.0-8.0 lbs	Triggers passport stamp: Master Angler
Ice Perch	Uncommon (Winter)	18 coins	1.5-4.0 lbs	Seasonal; Winter only — 90-day rotation

Species	Rarity	Coin Reward	Weight Range	Notes
Sunfish	Uncommon (Summer)	18 coins	1.5-4.0 lbs	Seasonal; Summer only — 90-day rotation

Tension Mechanic Design

- Sweet spot tension increases **35%/second** when pointer is inside the target zone
- Tension drops **20%/second** when pointer is outside the target zone
- Fish "rage runs" telegraph with a visual **"!"** appearing **0.4 seconds** early, giving the player reaction time
- Variable fish weight (0.5-8.0 lbs) directly scales fight duration and applies a coin multiplier
- Fishing Rod upgrades (Starter → River → Master) widen sweet spot and raise max tension threshold

Fishing Leaderboard & Tournaments

- **Weekly leaderboard:** top 10 catches by weight reset every Monday 00:00 UTC; winner earns "Master Angler" passport stamp
- **Saturday tournaments:** server-scheduled 1-hour sprints where all catches count; winner earns special HavenGem reward
- **Fish Collection Log:** passport-linked personal log tracking species caught, personal best weight per species, and total fish count

New Prisma Schema

```
model FishCatch {
  id          String @id @default(cuid())
  userId      String
  species     String
  weightLbs   Float
  coinsEarned Int
  caughtAt    DateTime @default(now())
  user        User    @relation(fields: [userId], references: [id])
}
```

```

model FishingLeaderboard {
  id      String @id @default(cuid())
  userId  String
  weightLbs Float
  species String
  weekOf  DateTime
  user    User
  @relation(fields: [userId], references: [id]) @@unique([userId, weekOf])
}

```

Socket.io Event Contract

Direction	Event	Payload
Client → Server	CAST_LINE	{ roomId }
Client → Server	REEL_POSITION	{ value: 0.0–1.0 }
Client → Server	CANCEL_FISHING	<i>none</i>
Server → Client	FISH_BITE	{ species, difficulty }
Server → Client	TENSION_UPDATE	{ value, sweetSpotMin, sweetSpotMax }
Server → Client	FISH_CAUGHT	{ species, weight, coins, isRare }
Server → Client	FISH_ESCAPED	<i>none</i>

Implementation Steps

1. **MAC** Create `src/shared/fishing.ts` with fish catalog, tension constants, and weight formula.
2. **MAC** Create `src/client/fishing/FishingScene.ts` — Babylon.js overlay panel with tension bar UI using `BABYLON.GUI.Rectangle` and `BABYLON.GUI.Slider`.
3. **LINUX** Create `src/server/fishing.ts` — server-authoritative catch validation (client sends reel positions, server determines outcome) to prevent cheating.
4. **LINUX** Add FishCatch and FishingLeaderboard tables via Prisma migration: `prisma migrate dev --name add_fishing`
5. **LINUX** Register Socket.io handlers in the fishing namespace in `server/index.ts`.
6. **MAC** Add "Fishing Dock" interaction trigger near Haven Park water spot in the Babylon.js room scene (invisible trigger mesh).

7. **LINUX** Add weekly leaderboard reset cron job using `node-cron`: resets every Monday 00:00 UTC.
8. **LINUX** PM2 reload after all server changes: `pm2 reload havenworld`

\$2 Loft Privacy Modes & Doorbell System

File references: `src/server/rooms/privacy.ts` `tests/loft-permissions.test.mjs`

Personal lofts have four access modes and a real-time doorbell system for controlled visitor entry. All access mode enforcement is server-side — the client cannot bypass room locks by manipulating socket events.

Access Mode Definitions

Mode	Behavior	Doorbell
Public	Anyone enters freely, no check required	Disabled (not needed)
Friends-Only	Friends list checked server-side against RoomManager state	Disabled
Password-Protected	Owner sets a 4-16 character passphrase; bcrypt hashed in DB	Enabled (alternative to password)
Locked	Owner-only access; no visitors admitted	Disabled

Doorbell Flow & Knock Queue

- Visitor clicks “Knock” → `RING_DOORBELL` emitted → owner receives toast notification with visitor avatar thumbnail and name
- Owner clicks **Admit** → visitor teleported into loft | Owner clicks **Decline** → visitor sees “The owner is busy” message
- Knock queue stored in Redis: `SETEX knock:{roomId}:{visitorId} 120 "pending"` — up to 5 pending knocks with 120-second TTL

- If owner ignores for 120 seconds, auto-declines and visitor is notified
- **Loft Away Mode:** owner sets an away message (max 80 chars) displayed to knockers when they ring

Co-Decorator Rights

Owner can promote up to **3 friends** to “Decorator” role for their loft, granting furniture placement/removal rights in that loft only. Stored in a `RoomDecorators` junction table. Access log records the last 20 visitors (name, timestamp) visible to the owner in the loft settings panel.

New Prisma Schema

```
enum RoomPrivacy { PUBLIC FRIENDS_ONLY PASSWORD_PROTECTED LOCKED }
model RoomDecorator {
  id      String @id @default(cuid())
  roomId  String
  userId  String
  grantedAt DateTime @default(now())
  room    Room
  @relation(fields: [roomId], references: [id])
  user    User
  @relation(fields: [userId], references: [id])
  @@unique([roomId, userId])
}
model RoomAccessLog {
  id      String @id @default(cuid())
  roomId  String
  visitorId String
  visitedAt DateTime @default(now())
  room    Room
  @relation(fields: [roomId], references: [id])
}
// Add to Rooms model:
// privacy RoomPrivacy @default(PUBLIC)
// passwordHash String?
// awayMessage String?
```

Socket.io Event Contract

Direction	Event	Payload
Client → Server	RING_DOORBELL	{ roomId, visitorId }
Client → Server	DOORBELL_DECISION	{ roomId, visitorId, admit: boolean }
Client → Server	SET_ROOM_PRIVACY	{ roomId, mode, password? }

Direction	Event	Payload
Client → Server	GRANT_DECORATOR	{ roomId, targetUserId }
Client → Server	REVOKE_DECORATOR	{ roomId, targetUserId }
Server → Client	DOORBELL_RING	{ visitorId, visitorName, avatarUrl }
Server → Client	DOORBELL_RESULT	{ admitted: boolean, message? }
Server → Client	PRIVACY_UPDATED	{ roomId, mode }

Implementation Steps

9. **LINUX** Add `RoomPrivacy` enum and new fields (`privacy`, `passwordHash`, `awayMessage`) to Rooms model in `schema.prisma`. Run: `pnpm prisma migrate dev --name add_room_privacy`
10. **LINUX** Create `RoomDecorator` and `RoomAccessLog` tables in the same migration.
11. **LINUX** Create `src/server/rooms/privacy.ts` — `PrivacyManager` class:


```
checkAccess(userId, roomId, hashPassword(pass), validatePassword(input, hash) using bcrypt.
```
12. **LINUX** Add doorbell knock queue in Redis: `SETEX knock:{roomId}:{visitorId} 120 "pending"`
13. **LINUX** Register Socket.io handlers in `RoomManager` for all doorbell and privacy events.
14. **MAC** Add loft settings panel in client UI: privacy dropdown, password field, decorator list, and away message field.
15. **MAC** Add “Knock” button to loft entrance in Babylon.js room scene — only visible when the player is not the room owner.
16. **LINUX** Write Jest tests in `tests/loft-permissions.test.mjs` covering all 4 access modes.

§3 Loft Guestbook & Tip Jar

File reference: `src/server/guestbook.ts`

A social layer for personal lofts. Visitors can sign the guestbook and tip the owner with HavenCoins, creating positive social reciprocity and reasons to visit other players' lofts regularly.

Guestbook Design

- In-world 3D guestbook object (open book mesh on a stand) in each loft — click to open a `BABYLON.GUI` panel
- Visitor writes a message up to **120 characters**, stored with avatar name, avatar snapshot thumbnail URL, and timestamp
- Pagination: 10 entries per page, newest first; owner can delete individual entries
- Messages filtered server-side through **bad-words** npm package before saving
- Owner receives a daily digest in the in-game notification bell showing new guestbook entries since last login

Tip Jar Design

- 3D coin jar mesh near loft entrance; clicking opens an amount-selection modal
- Tip increments: **10, 25, 50, 100, 250, 500** HavenCoins
- Server validates sender has sufficient coins before transfer — uses a Prisma transaction for atomicity
- Owner receives real-time Socket.io notification: *"PlayerName tipped you 50 HavenCoins! 📁"*
- Tip history: owner views last 30 tips (date, tipper name, amount) in a settings panel

New Prisma Schema

```
model GuestbookEntry { id      String @id @default(cuid()) roomId    String
authorId  String message    String @db.VarChar(120) authorName String
authorAvatar String? createdAt DateTime @default(now()) room      Room
@relation(fields: [roomId], references: [id]) author    User @relation(fields:
[authorId], references: [id]) } model TipTransaction { id      String @id
@default(cuid()) senderId String receiverId String amount    Int roomId
String createdAt DateTime @default(now()) sender    User
@relation("TipSender", fields: [senderId], references: [id]) receiver User
@relation("TipReceiver", fields: [receiverId], references: [id]) }
```

Socket.io Event Contract

Direction	Event	Payload
Client → Server	SIGN_GUESTBOOK	{ roomId, message }
Client → Server	TIP_OWNER	{ roomId, amount }
Client → Server	GET_GUESTBOOK	{ roomId, page }
Client → Server	DELETE_GUESTBOOK_ENTRY	{ entryId }
Server → Client	GUESTBOOK_PAGE	{ entries: [], totalPages }
Server → Client	GUESTBOOK_SIGNED	{ entry }
Server → Client	TIP_RECEIVED	{ amount, senderName }
Server → Client	TIP_SUCCESS	{ newBalance }
Server → Client	TIP_ERROR	{ message }

Implementation Steps

17. **LINUX** Add `GuestbookEntry` and `TipTransaction` models to `schema.prisma`. Run:

```
npx prisma migrate dev --name add_guestbook_tipjar
```

18. **LINUX** Install bad-words package: `npm install bad-words`
19. **LINUX** Create `src/server/guestbook.ts` — `GuestbookService` with `ignBook()`, `deleteEntry()`, `getPage()`, `ipOwner()`. The `ipOwner()` method uses a Prisma transaction to atomically deduct coins from sender and credit receiver.
20. **LINUX** Register Socket.io handlers in `server/index.ts`.
21. **MAC** Create Babylon.js interactive guestbook mesh (open book model) in loft room scenes — clicking opens a `BABYLON.GUI` panel.
22. **MAC** Create tip jar mesh interaction near loft entrance — clicking opens amount-selection modal.
23. **MAC** Add coin balance display in UI header that updates in real-time via `TIP_SUCCESS` event.

§4 Procedural Zero-Asset Sound Engine

File reference: `src/client/shared/audio.ts`

A zero-download sound system using the browser's Web Audio API (`OscillatorNode` + `GainNode`) to synthesize all game audio procedurally. No `.mp3` or `.ogg` files are ever served — eliminating all audio network overhead entirely.

Why Procedural Audio Matters

In a browser-based social game, audio file loading is a significant source of page-load latency. By synthesizing all sounds via the Web Audio API, HavenWorld achieves instant audio with no network requests — and a unique sonic identity impossible to reproduce from a generic sound library.

Sound Catalog

Sound ID	Technique	Parameters	Trigger
footstep	Sine burst	80 Hz, 40ms, every 350ms while moving	Avatar movement loop
coin_pickup	Triangle wave chime	E5 (659 Hz) → B5 (988 Hz), 80ms each, ConvolverNode reverb	COINS_EARNED event
furniture_place	Sine decay pop	200 Hz → 50 Hz over 80ms	Furniture placement confirmed
chat_message	Sine envelope	880 Hz, 5ms attack, 200ms decay	Incoming chat message
room_whoosh	Filtered noise	White noise, BiquadFilter lowpass Q=2, 800→200 Hz over 400ms	Room transition
doorbell	Square wave chime	C5 → G5, 150ms each, soft gain	DOORBELL_RING event
fishing_bite	Ascending arpeggio	E4, G4, B4 — 60ms each	FISH_BITE event
fishing_catch	Triangle fanfare	C5, E5, G5, C6 — 100ms each	FISH_CAUGHT event
pet_happy	Sine squeak	C6 → E6, 60ms	PET_REACT event
trade_complete	Sharp sine ding	1047 Hz (C6), 20ms attack, 300ms decay	TRADE_COMPLETE event

User Preference System

- Master volume control (0-1) stored in `localStorage`
- Mute toggle persisted across sessions
- Per-category toggles in settings panel: Movement, UI, Social, Game Events
- `AudioContext` lazy-initialized on first user gesture to comply with browser autoplay policies (critical for Safari)

- Settings object persisted as `havenworld_audio_settings` JSON in `localStorage`

Implementation Steps (All MAC)

24. **MAC** Create `src/client/shared/audio.ts` — `AudioEngine` class with lazy `AudioContext` initialization gated on first user gesture.
25. **MAC** Implement `SoundCatalog` object mapping sound names to synthesis functions (each a pure function returning an `AudioBuffer` or triggering `OscillatorNode` directly).
26. **MAC** Add `playSound(name: string)` public method — checks mute state, applies master volume via `GainNode`, triggers the appropriate synthesis function.
27. **MAC** Add `AudioSettings` panel to game UI: master volume slider, mute button, per-category toggles.
28. **MAC** Persist settings to `localStorage`: `havenworld_audio_settings` JSON object.
29. **MAC** Wire sound triggers throughout the client: footstep timer in avatar movement loop, coin pickup on `COINS_EARNED`, doorbell on `DOORBELL_RING`, etc.
30. **MAC** Test `AudioContext` unlock on mobile (Safari requires a user gesture) — add a one-time `click` handler on the `Babylon.js` canvas element.

§5 Anti-Scam P2P Trading System

File references: `src/server/trade.ts` `src/client/trade/TradeModal.ts`

A two-player item and HavenCoin trading system with multi-stage confirmation designed from the ground up to prevent the four classic scam patterns: bait-and-switch, race condition exploitation, disconnect-during-transfer, and slot manipulation after the ready lock.

Trade State Machine

State	Description	Exit Conditions
IDLE	No active trade	Player sends TRADE_REQUEST
REQUESTED	Awaiting target acceptance	TRADE_ACCEPT or TRADE_DECLINE
ACCEPTED	Both players in trade window	Either begins adding items
OFFER_PHASE	Both sides adding items and coins	Both click Ready
LOCKED	Slots locked, no changes allowed	Both click Confirm (5s countdown)
CONFIRMED	Both confirmed, executing swap	Prisma transaction completes
COMPLETED	Trade successful, audit log written	Terminal state
CANCELLED	Trade cancelled at any step	Terminal state — both notified

Anti-Scam Design Rules

- **Proximity enforcement:** both players must be in the same room, within 3 tile-units of each other — validated server-side using RoomManager position data
- **Two-stage lock:** Stage 1 — both click “Ready” (slots lock). Stage 2 — both click “Confirm” within a 5-second countdown. Any slot change resets both to unready
- **Atomic transfer:** Prisma `$transaction()` ensures both inventories update or neither does; includes rollback on connection drop
- **Trade audit log:** every completed trade stored in `TradeLog` with timestamp, both player IDs, all items and coins — retained 30 days for dispute resolution
- **Trade cooldown:** 30-second cooldown between trades with the same partner to prevent spam
- **Offer limit:** up to 6 inventory items per side + optional HavenCoin amount (0-9,999)

New Prisma Schema

```
enum TradeStatus { PENDING COMPLETED CANCELLED } model TradeLog {
  id          String    @id @default(cuid()) initiatorId  String receiverId  String
  initiatorItems Json    receiverItems Json    initiatorCoins Int      @default(0)
  receiverCoins Int      @default(0) status      TradeStatus createdAt
  DateTime @default(now()) completedAt  DateTime? initiator    User
  @relation("TradeInitiator", fields: [initiatorId], references: [id]) receiver    User
  @relation("TradeReceiver", fields: [receiverId], references: [id]) }
```

Socket.io Event Contract

Direction	Event	Payload
Client → Server	TRADE_REQUEST	{ targetUserId }
Client → Server	TRADE_ACCEPT	<i>none</i>
Client → Server	TRADE_DECLINE	<i>none</i>
Client → Server	OFFER_ITEM	{ slotIndex, inventoryItemId }
Client → Server	OFFER_COINS	{ amount }
Client → Server	TRADE_READY	<i>none</i>
Client → Server	TRADE_CONFIRM	<i>none</i>
Client → Server	TRADE_CANCEL	<i>none</i>
Server → Client	TRADE_REQUESTED	{ initiatorName, initiatorId }
Server → Client	TRADE_STATE_UPDATE	{ state, offers }
Server → Client	TRADE_COUNTDOWN	{ seconds }
Server → Client	TRADE_COMPLETE	{ summary }
Server → Client	TRADE_CANCELLED	{ reason }

Implementation Steps

31. **LINUX** Add `TradeLog` model to `schema.prisma`. Run: `npm run prisma migrate dev -- --name add_trade_system`
32. **LINUX** Create `src/server/trade.ts` — `TradeManager` class with `Map<string, TradeSession>` for in-memory active trades and Redis TTL for session cleanup.
33. **LINUX** Implement atomic swap in `TradeSession.complete()` using `prisma.$transaction()`.
34. **LINUX** Add proximity check in `TRADE_REQUEST` handler using current room position data from `RoomManager`.
35. **MAC** Create `src/client/trade/TradeModal.ts` — Babylon.js GUI trade window: two columns (My Offer / Their Offer), 6 item slots each, coin input, Ready/Confirm/Cancel buttons.
36. **MAC** Add right-click context menu on avatars in room scene with “Trade” option — emits `TRADE_REQUEST`.
37. **LINUX** Write test coverage for the 4 classic scam patterns: bait-and-switch, race condition, disconnect-during-transfer, and slot manipulation after Ready.

§6 Dual-Currency Economy & 48-Hour Rotating Shop

File references: `src/shared/catalog.ts` `src/server/shop.ts`

HavenWorld runs a two-currency economy designed for long-term health: a freely-earned soft currency for daily engagement, and a prestige currency earned through genuine accomplishment — never purchased.

Currency Design

Currency	Icon	How Earned	How Spent
HavenCoins		Fishing, Pizza	Common catalog

Currency	Icon	How Earned	How Spent
		Chef, daily login streak, tip jars received, selling items, event participation	items, furniture, crafting materials, club creation, loft name changes
HavenGems		Achievements, weekly leaderboard wins, seasonal event rewards, Legendary fishing catches	Exclusive cosmetics, rare furniture, pet accessories, avatar accessories, passport frames

HavenGems are Never Purchased

HavenGems are earned entirely through gameplay achievement — they are never available for real-money purchase. This keeps HavenWorld entirely free-to-play competitive while still maintaining a prestigious prestige layer accessible to all dedicated players.

48-Hour Epoch Rotation Algorithm

The featured shop rotates every 48 hours using a deterministic seeded shuffle:

```
epochNumber = Math.floor(Date.now() / 172_800_000) // Seed a Fisher-Yates shuffle of the featured item pool with epochNumber // Result is identical on all clients and the server – no database dependency
```

This means every player worldwide sees the same 3 featured items at the same time, with a synchronized countdown timer to the next rotation — creating shared social moments around shop refreshes.

Daily Login Streak Rewards

Day Milestone	HavenCoins	HavenGems	Bonus
Day 1	10	—	—
Day 3	25	—	—

Day Milestone	HavenCoins	HavenGems	Bonus
Day 7	100	1	—
Day 14	200	3	—
Day 30	500	10	“Loyal Haven” passport badge

Economy Sinks (Coin Drains)

- Furniture placement fee: **2 HavenCoins per piece**
- Loft name change: **50 HavenCoins**
- Club creation fee: **500 HavenCoins**
- Passport frame cosmetics: **HavenGems only**
- Seasonal limited items: **HavenGems only**

New Prisma Schema

```
model DailyLoginStreak {
  id          String @id @default(cuid())
  userId      String @unique
  currentStreak Int   @default(0)
  longestStreak Int   @default(0)
  lastLoginDate DateTime
  user        User    @relation(fields: [userId], references: [id])
}
model GiftTransaction {
  id          String @id @default(cuid())
  senderId    String
  receiverId  String
  itemId      String
  message     String?
  createdAt   DateTime @default(now())
  sender      User
  @relation("GiftSender", fields: [senderId], references: [id])
  receiver    User
  @relation("GiftReceiver", fields: [receiverId], references: [id])
}
// Add to Users
model User {
  havenGems Int @default(0)
```

Socket.io Event Contract

Direction	Event	Payload
Client → Server	BUY_ITEM	{ itemId, currency }
Client → Server	GET_SHOP	<i>none</i>
Client → Server	CLAIM_DAILY_LOGIN	<i>none</i>
Client → Server	GIFT_ITEM	{ itemId, recipientId, message }
Server → Client	SHOP_STATE	{ featured[], permanent[], epochEnds }
Server → Client	PURCHASE_SUCCESS	{ newBalance, item }
Server → Client	PURCHASE_ERROR	{ reason }
Server → Client	DAILY_REWARD	{ coins, gems, streak, badge? }
Server → Client	FLASH_SALE_START	{ item, discountPct, expiresAt }
Server → Client	GIFT_RECEIVED	{ item, senderName, message }

Implementation Steps

38. **LINUX** Add `havenGem` to Users, add `dailyLoginStreak` and `giftTransaction` models. Run: `npm run prisma migrate dev --name add_economy`
39. **LINUX** Create `src/shared/catalog.ts` — full typed item catalog array. Each item: `{ id, name, category, priceCoin, priceGem, rarity, featured: boolean }`
40. **LINUX** Create `src/server/shop.ts` — `ShopService`: `getFeatured()` using seeded Fisher-Yates shuffle with epoch seed, `buyItem()`, `claimDailyLogin()`, `giftItem()`.
41. **LINUX** Add flash sale scheduler: `node-cron` job triggering random flash sales 2-3x per day via Redis pub/sub → Socket.io broadcast.
42. **MAC** Build shop UI panel in Babylon.js: tabbed interface (Featured / Catalog), item cards with thumbnails, buy buttons, currency balance display.
43. **MAC** Build daily login modal: auto-displays on first login each day, shows streak progress and claimed reward.

44. **LINUX** Add admin economy monitoring endpoint: `GET /admin/economy-stats` showing total coins in circulation, average daily earnings, and top spenders.

§7 Pet Companions & Autonomous AI

File references: `src/shared/pet.ts` `src/client/pets/PetController.ts`
`src/server/pets.ts`

Adoptable pet companions (cats, dogs, baby dragons) with autonomous behavior that follows and reacts to their owner in 3D rooms. Each pet type has a unique personality expressed through a finite state machine that ticks server-side every 2 seconds.

Pet Types & Personalities

Pet Type	Personality	Default Behavior	Unique Trait
Cat	Playful, independent	Alternates wander and follow	Sits and ignores owner more often
Dog	Loyal, attentive	Always follows owner	Never enters WANDER state unprompted
Baby Dragon	Excitable, reactive	Follows with frequent REACT states	Loudest visual reactions; reacts to everything

Pet Finite State Machine

State	Duration	Condition to Transition
IDLE	3-10 seconds random	Timeout → WANDER or FOLLOW
WANDER	Until destination reached	Reached point → IDLE
FOLLOW	While owner moving	Owner idle 60s → SLEEP

State	Duration	Condition to Transition
SLEEP	Until owner moves	Owner enters room or moves → REACT
REACT	1-2 seconds	Triggered by owner action (coins, chat, room enter) → IDLE

Pet Stats

- **Happiness (0-100):** decreases 2/hour when not in same room as owner; increases on owner interaction. Affects follow frequency and wander rate
- **Hunger (0-100):** decreases 5/hour. Feed with Pet Treats (HavenCoin shop item). Hungry pet shows 🍖 emote and stops following
- **Visual reactions:** floating billboard sprites (♥, !, 🛌, 🎵) using `BABYLON.GUI.Rectangle` positioned in world space
- **Social greeting:** pets of two adjacent players “greet” each other with a meeting animation if both owners are friends

New Prisma Schema

```
enum PetType { CAT DOG BABY_DRAGON } enum PetState { IDLE
WANDER FOLLOW SLEEP REACT } model Pet { id      String @id
@default(cuid()) ownerId   String petType   PetType name      String
@db.VarChar(16) happiness  Int    @default(100) hunger      Int
@default(100) accessoryIds Json   @default("[]") lastFedAt   DateTime
@default(now()) createdAt  DateTime @default(now()) owner     User
@relation(fields: [ownerId], references: [id]) }
```

Socket.io Event Contract

Direction	Event	Payload
Client → Server	ADOPT_PET	{ petType }
Client → Server	NAME_PET	{ petId, name }
Client → Server	FEED_PET	{ petId }
Client → Server	EQUIP_PET_ACCESSORY	{ petId, itemId }
Server → Client	PET_STATE_UPDATE	{ petId, state, position, targetPosition }
Server → Client	PET_REACT	{ petId, emote }
Server → Client	PET_HUNGER_LOW	{ petId }
Server → Client	PET_ADOPTED	{ pet }

Implementation Steps

45. **LINUX** Add `Pet` model to `schema.prisma`. Run: `npx prisma migrate dev --name add_pets`
46. **LINUX** Create `src/server/pets.ts` — `PetManager` class, ticks pet state machines every 2 seconds using `setInterval`, broadcasts `PET_STATE_UPDATE` to all players in the room.
47. **MAC** Create `src/client/pets/PetController.ts` — receives state updates, animates pet GLB model using `Babylon.js AnimationGroup`, moves pet toward `targetPosition` using smooth lerp.
48. **MAC** Install and configure RecastJS for pet pathfinding around furniture: `npm install @babylonjs/recast`
49. **MAC** Create pet billboard system: `BABYLON.GUI.StackPane` attached to pet mesh in `WorldSpace`, showing pet name and emotion emote on `REACT` event.
50. **LINUX** Add hourly `node-cron` job to decrease all active pet happiness and hunger stats in the database.
51. **MAC** Build Pet Management panel: shows owned pets, stats (happiness bar, hunger bar), name field, accessory slots, and Feed button.

\$8 Pizza Chef Mini-Game

File references: `src/shared/pizza.ts` `src/client/minigame/PizzaScene.ts`
`src/server/minigame.ts`

A job mini-game where players assemble pizza orders against a timer to earn HavenCoins. Drag-and-drop ingredient assembly in a dedicated full-screen GUI overlay. A weekly earnings cap of 2,000 coins maintains economy balance.

Recipe Definitions

Recipe	Ingredient Order	Timer	Unlocked After
Margherita	Dough → Sauce → Mozzarella	15 seconds	Default (always available)
Funghi Rustica	Dough → Sauce → Mushrooms → Mozzarella	20 seconds	10 Perfect Margheritas
Green Garden	Dough → Basil Pesto → Zucchini → Spinach	22 seconds	10 Perfect Funghi Rusticas
Seasonal Special	Varies by season (e.g., Pumpkin Harvest in Autumn)	25 seconds	Active seasonal event; double coin reward

Scoring System

Tier	Condition	Coins Awarded
Perfect	Correct ingredient order, within timer	30 coins
Good	Correct but slightly slow	20 coins
Fail	Wrong ingredient or time expired	5 coins + retry

A **3x consecutive Perfect** combo grants a $\times 1.5$ coin multiplier for the next 5 orders. A 5-minute shift completion earns a bonus of **+50 coins** if all orders during the shift were Perfect.

New Prisma Schema

```
model MinigameSession { id      String @id @default(cuid()) userId  String
gameType  String coinsEarned Int  score    Int  duration  Int  completedAt
DateTime @default(now()) user     User   @relation(fields: [userId], references:
[id]) } model WeeklyEarningsCap { id      String @id @default(cuid()) userId
String gameType String earned  Int    @default(0) weekOf  DateTime user
User   @relation(fields: [userId], references: [id]) @@unique([userId, gameType,
weekOf]) }
```

Implementation Steps

52. **LINUX** Add `MinigameSession` and `WeeklyEarningsCap` models. Run: `npm prisma migrate dev --name add_minigames`
53. **LINUX** Create `src/server/minigame.ts` — `MinigameService: validatePizzaOrder()`, `awardCoins()` with weekly cap enforcement, `recordSession()`.
54. **MAC** Create `src/client/minigame/PizzaScene.ts` — dedicated Babylon.js full 2D GUI overlay panel (not a 3D scene) for the pizza assembly interface.
55. **MAC** Implement drag-and-drop ingredient system using `BABYLON.GUI.Pointer` events.
56. **MAC** Build timer bar using `BABYLON.GUI.Rectangle` with animated width reduction each frame.
57. **LINUX** Register Socket.io `PIZZA_ORDER_SUBMIT` handler — validates server-side and returns coin award.

58. **MAC** Add Pizza Chef job station in the Haven Hub room: a counter mesh with interaction trigger that launches `PizzaScene`.

§9 Workshop Crafting & Furniture Recycling

File references: `src/shared/crafting.ts` `src/server/workshop.ts`

Players recycle unwanted furniture into raw materials and craft exclusive Workshop-only items. The Workshop creates an economy loop for unwanted inventory items and provides an aspirational crafting progression ladder.

Raw Material Types

Material	Sourced From	Recycling Yield Formula
scrap_metal	Metal/tech furniture	<code>Math.floor(item.priceCoin * 0.4 / materialValue)</code>
timber	Wooden furniture	Same formula
fabric	Soft furnishings, rugs	Same formula
crystal_shard	Decorative/gem items	Same formula

Crafting Recipes (Workshop Exclusive)

Recipe	Materials Required	Rarity	Craft Time
Reclaimed Bookshelf	10 timber + 2 fabric	Common	15 minutes
Steampunk Sofa	8 scrap_metal + 3 timber	Rare	45 minutes
Clockwork Clock	6 scrap_metal + 2 crystal_shard	Rare	45 minutes
Neon Lamp	4 crystal_shard + 2 scrap_metal	Rare	45 minutes

Recipe	Materials Required	Rarity	Craft Time
Dragon Egg Planter	5 crystal_shard + 3 timber	Legendary	2 hours
Snowglobe Lamp (Winter)	Seasonal recipe — Christmas event only	Rare	30 minutes
Haunted Mirror (Halloween)	Seasonal recipe — Autumn event only	Rare	30 minutes

New Prisma Schema

```

model MaterialInventory {
  id          String @id @default(cuid())
  userId      String
  @unique
  scrapMetal  Int    @default(0)
  timber      Int    @default(0)
  fabric      Int    @default(0)
  crystalShard Int    @default(0)
  user        User   @relation(fields: [userId], references: [id])
}

model CraftingQueue {
  id          String @id
  @default(cuid())
  userId      String
  recipeId    String
  startedAt   DateTime @default(now())
  completesAt DateTime
  claimed     Boolean @default(false)
  user        User   @relation(fields: [userId], references: [id])
}

```

Socket.io Event Contract

Direction	Event	Payload
Client → Server	RECYCLE_ITEM	{ inventoryItemId }
Client → Server	RECYCLE_BULK	{ inventoryItemIds[] }
Client → Server	START_CRAFT	{ recipeId }
Client → Server	CLAIM_CRAFT	{ craftingQueueId }
Server → Client	RECYCLE_RESULT	{ materials }
Server → Client	CRAFT_STARTED	{ completesAt, recipeId }

Direction	Event	Payload
Server → Client	CRAFT_COMPLETE	{ item }
Server → Client	MATERIALS_UPDATE	{ scrapMetal, timber, fabric, crystalShard }

Implementation Steps

59. **LINUX** Add `MaterialInventory` and `CraftingQueue` models. Run: `npmx prisma migrate dev --name add_workshop`
60. **LINUX** Create `src/shared/crafting.ts` — typed `const` recipe array: `{ id, name, materials: { type, qty }[], outputItemId, craftingMinutes, rarity }`
61. **LINUX** Create `src/server/workshop.ts` — `WorkshopService`: `recycleItem()`, `bulkRecycle()`, `startCraft()` (validates materials, creates `CraftingQueue` entry), `checkCompletions()` run every 5 minutes via `node-cron`.
62. **LINUX** Register Socket.io handlers for all workshop events in `server/index.ts`.
63. **MAC** Build Workshop UI: tabbed interface — Recycle tab (inventory grid with material preview), Craft tab (recipe list, progress bars for active crafts).
64. **MAC** Add “Workshop” room to the Babylon.js client room list — a themed industrial/steampunk room scene.

\$10 Player Passport & Achievement Stamps

File references: `src/client/shared/passport.ts` `src/server/achievements.ts`

An in-game passport card summarizing a player's identity, VIP status, join date, badges, and achievement stamps. Viewable by other players to encourage social exploration and aspiration.

All 20 Achievement Stamps

Stamp Name	Requirement	Type
First Step	Log in for the first time	Milestone
World Traveler	Visit 10 different rooms	Exploration
Master Angler	Catch a Radiant Haven Koi (Legendary)	Mini-Game
Friendly Spirit	Make 5 friends	Social
Sanctuary Decorator	Place 20 pieces of furniture in your loft	Creative
Social Butterfly	Sign 10 guestbooks	Social
Tip Royale	Tip other players 1,000 coins total	Social
Craft Master	Craft 20 workshop items	Crafting
Pizza Pro	Earn 500 coins from Pizza Chef	Mini-Game
Fish and Chips	Catch 50 total fish	Mini-Game
Streaker	Maintain a 7-day login streak	Loyalty
Gem Hunter	Earn 25 HavenGems total	Economy
Pet Parent	Adopt a pet	Companion
Dragon Tamer	Adopt a Baby Dragon specifically	Companion
Top Chef	Finish in top 10 weekly Pizza Chef leaderboard	Mini-Game
Generous Soul	Send 5 gifts to other players	Social
Good Neighbor	Admit 10 doorbell visitors to your loft	Social
Loyal Haven	Maintain a 30-day login streak	Loyalty
Trading Post	Complete 10 P2P trades	Economy
Legend of Haven	Earn all other 19 stamps	Prestige

New Prisma Schema

```

model Achievement { id String @id @default(cuid()) userId String stamp
String earnedAt DateTime @default(now()) user User @relation(fields:
[userId], references: [id]) @@unique([userId, stamp]) } model PassportFrame { id
String @id @default(cuid()) userId String @unique frameId String
@default("default") user User @relation(fields: [userId], references: [id]) } //
Add to Users model: // isVIP Boolean @default(false) // vipSince DateTime?

```

Implementation Steps

65. **LINUX** Add `Achievement` and `PassportFrame` models, add `isVIP` and `vipSince` to `Users`. Run: `px prisma migrate dev --name add_passport_achievements`
66. **LINUX** Create `src/server/achievements.ts` — `AchievementService.checkAndAward(userId, action)` called after every relevant player action. Emits `ACHIEVEMENT_UNLOCKED` to user socket.
67. **MAC** Create `src/client/shared/passport.ts` — `PassportModal` class: renders 3D avatar portrait using `RenderTargetTexture` (256×256), lays out the 4×5 stamp grid, animates stamp unlock with a golden glow effect.
68. **MAC** Add achievement unlock toast notification: bottom-center of screen, showing stamp icon and name, 3-second display with fade in/out animation.
69. **LINUX** Expose `GET /api/passport/:userId` endpoint returning public passport data (no sensitive fields).
70. **MAC** Add “Inspect Passport” option to avatar right-click context menu in room scenes.

§11 Loft Ambient Moods

File references: `src/server/rooms.ts` (`SET_ROOM_MOOD` handler)
`src/client/rooms/MoodSystem.ts`

Lighting mood presets that the loft owner can apply to change the atmosphere of their room in real-time for all visitors. All 8 mood changes are validated server-side to ensure only the room owner can change mood.

Mood Preset Definitions

Mood ID	Ambient Color	Intensity	Special FX
day	Warm white (#ffffff)	1.0	Clear sky background
dusk	Amber/orange (#FF8C42)	0.6	Long-shadow directional light
night	Deep blue (#1a1a4e)	0.3	Point lights from lamp meshes only; star particle system
cyber_neon	Dark (#0a0a0a)	0.2	Neon emissive mesh highlights; DefaultRenderingPipeline bloom enabled
golden_hour	Gold (#FFD700)	0.8	Soft volumetric fog via BABYLON.FogMode.EXP2
haunted	Desaturated green (#2d4a2d)	0.25	Slow particle dust motes; flickering light animation
arctic	Ice blue (#cce8ff)	0.9	Snowflake particle system; slight blue fog
cozy_evening	Warm orange-red (#c47a3d)	0.5	Fireplace point light flicker animation

Mood changes tween over **2 seconds** using `BABYLON.Animation` on `Light.intensity` and `Light.diffuse` color (ANIMATIONTYPE_COLOR3, 30 fps). Persisted in `Rooms.moodPreset` and reapplied on room load for new visitors.

Socket.io Event Contract

Direction	Event	Payload
Client → Server	SET_ROOM_MOOD	{ roomId, mood: "day" "dusk" "night" "cyber_neon" "golden_hour" "haunted" "arctic" "cozy_evening" }
Server → Client	ROOM_MOOD_CHANGED	{ mood } — broadcast to all room occupants

Implementation Steps

71. **LINUX** Add `moodPreset String @default("day")` field to Rooms model. Run: `npmx prisma migrate dev --name add_room_mood`
72. **LINUX** Add `SET_ROOM_MOOD` Socket.io handler in `RoomManager` — validates sender is room owner, saves to DB, broadcasts `ROOM_MOOD_CHANGED` to all room occupants.
73. **MAC** Create `src/client/rooms/MoodSystem.ts` — `MoodPreset` constant with all 8 mood configs: `[ ambientColor, fogColor, fogDensity, lightIntensity, lightColor, bloomEnabled ]`
74. **MAC** Implement `applyMood(presetName)` function using `BABYLON.Animation` for smooth 2-second transitions.
75. **MAC** Add mood selector panel to loft owner controls: 8 colored swatches with preset name labels.
76. **MAC** Wire each mood to its ambient audio synthesis profile in `AudioEngine` (drone frequencies matching the visual atmosphere).

§12 Seasonal Events System ★ New Addition

A recurring calendar of themed seasonal events refreshing the game world every 6–8 weeks. Seasonal content is the #1 driver of returning player retention in social

games — Club Penguin's seasonal parties built loyal communities over a decade. HavenWorld needs this infrastructure from launch.

Annual Season Calendar

Season	Month	Duration	Temporary Room
Spring Bloom	March	3 weeks	Cherry Blossom Garden
Summer Splash	June	3 weeks	Beachside Boardwalk
Autumn Harvest	September	3 weeks	Pumpkin Patch Fields
Winter Wonderland	December	3 weeks	Winter Snowfield
New Year's Gala	January	2 weeks	Gala Ballroom
HavenDay Anniversary	Launch anniversary	2 weeks	Haven Historical Museum

Per-Season Feature Bundle

- 4–6 exclusive seasonal items in a limited shop (HavenCoins only, never HavenGems — must be accessible to all players)
- 2 seasonal achievements added to the Achievement system
- 1 seasonal fish variant added to the fishing rotation
- 1 seasonal pizza recipe unlocked in Pizza Chef
- 1 seasonal pet accessory available for HavenCoins
- Seasonal particle system in the temporary room (snowflakes, cherry blossoms, falling leaves)
- Seasonal currency (e.g., SnowFlakes during Winter) earned via event activities; converts to HavenCoins 1:1 at event end

New Prisma Schema

```
model SeasonalEvent { id String @id @default(cuid()) name String
  theme String startsAt DateTime endsAt DateTime isActive Boolean
  @default(false) rewards Json } model SeasonalProgress { id String @id
  @default(cuid()) userId String eventId String tier Int @default(0)
  currency Int @default(0) user User @relation(fields: [userId],
  references: [id]) event SeasonalEvent @relation(fields: [eventId], references: [id])
  @@unique([userId, eventId]) }
```

Implementation Steps

77. **LINUX** Add `SeasonalEvent` and `SeasonalProgress` models. Run: `npx prisma migrate dev --name add_seasonal_events`
78. **LINUX** Create `src/server/events/seasonal.ts` — `SeasonalEventService:`
`getActiveEvent()`, `grantSeasonalCurrency()`, `advanceTier()`.
79. **LINUX** Seed the database with all 6 seasonal event definitions including date ranges for the next 2 years using a Prisma seed script.
80. **MAC** Build seasonal HUD widget: shows current event banner, seasonal currency balance, tier progress bar, and time-remaining countdown.
81. **MAC** Add seasonal room variant loader in the client room system — checks `getActiveEvent()` on room load and overlays seasonal GLB and particle system.

§13 Player Club System ★ New Addition

Players create or join clubs (up to 50 members) with their own clubhouse room, private club chat, and weekly challenge leaderboard. Social bonds are the #1 long-

term retention factor in virtual worlds — clubs create community accountability and daily log-in reasons beyond personal gameplay.

Club Feature Set

- **Creation cost:** 500 HavenCoins (economy sink) — creates a Club with name (max 24 chars), motto (max 80 chars), and optional tag (max 5 chars, shown beside member names in chat)
- **Roles:** Owner (full control), Officer (invite/kick members), Member (participate)
- **Club chat:** persistent text channel with 48-hour Redis history backing
- **Clubhouse room:** dedicated private room instance for each club, furnishable by Officer-rank members
- **Weekly challenge:** shared collective goal (e.g., “Catch 100 total fish this week”) with progress bar; completion earns all members 100 bonus HavenCoins
- **Roster panel:** all members with online status indicator, last-seen date, and role badge
- **Limit:** maximum 1 club per player — must leave to join another

New Prisma Schema

```
enum ClubRole { OWNER OFFICER MEMBER }
model Club {
  id String @id @default(cuid())
  name String @unique
  @db.VarChar(24)
  motto String? @db.VarChar(80)
  tag String? @db.VarChar(5)
  ownerId String
  roomId String? @unique
  createdAt DateTime @default(now())
  owner User @relation(fields: [ownerId], references: [id])
  members ClubMember[]
}
model ClubMember {
  id String @id @default(cuid())
  clubId String
  userId String
  role ClubRole @default(MEMBER)
  joinedAt DateTime @default(now())
  club Club @relation(fields: [clubId], references: [id])
  user User
}
```

```
@relation(fields: [userId], references: [id]) @@unique([userId]) }
```

Implementation Steps

82. **LINUX** Add `Club` and `ClubMember` models to `schema.prisma`. Run: `npm run migrate dev --name add_clubs`
83. **LINUX** Create `src/server/clubs.ts` — `clubService: { createClub(), inviteMember(), kickMember(), getClubInfo(), updateClub() }`.
84. **LINUX** Add Club chat channel to Socket.io: `club:{clubId}` namespace with Redis 48-hour history backing using `L.PUSH / L.TRIM`.
85. **MAC** Build Club panel in client: tabbed (My Club, Find Clubs, Leaderboard). My Club tab shows roster, chat window, and weekly challenge progress bar.

§14 Emote Wheel & Expression System ★ New Addition

A radial emote wheel triggered by pressing **E** (or a dedicated mobile button) giving instant access to 12 avatar animations for expressive social interaction. Emotes are visible to all players in the room and accompanied by a floating icon bubble above the avatar's head.

Default Emote Set

Slot	Emote Name	Mixamo Animation	Duration
1	Wave	Wave Hip Hop	2.5s
2	Dance	Macarena	Loop (5s then stop)
3	Laugh	Laughing	2s
4	Cry	Crying	2s

Slot	Emote Name	Mixamo Animation	Duration
5	Shrug	Shrug	1.5s
6	Clap	Clapping	2s
7	Thumbs Up	Thumbs Up	1.5s
8	Bow	Bowing	2s
9	Sit	Sitting Idle	Hold until cancelled
10	Sleep	Sleeping	Hold until cancelled
11	Heart	Kiss	2s
12	Spin	Spin	1.5s

Socket.io Event Contract

Direction	Event	Payload
Client → Server	EMOTE_TRIGGERED	{ emoteId }
Server → Client	AVATAR_EMOTE	{ userId, emoteId } — broadcast to room

Implementation Steps

86. **MAC** Add 12 emote `AnimationGroup` assets to avatar GLB via Blender/Mixamo pipeline — additional Mixamo animations merged into the existing avatar GLB.
87. **MAC** Create `EmoteWheel` UI component: radial menu appearing on E key press using `BABYLON.GUI` circles arranged in a ring (12 positions at 30° intervals).
88. **LINUX** Register `EMOTE_TRIGGERED` Socket.io handler — validates `emoteId` against the allowed list, then broadcasts `AVATAR_EMOTE` to the room.
89. **MAC** Add emote icon bubble billboard above avatar in `AvatarController` — 3-second display via `BABYLON.GUI.Rectangle` positioned in world space, then fades out.

\$15 Daily Quest System ★ New Addition

Three rotating daily quests giving players concrete goals each session, boosting average session length and next-day return rate. Quest selection is deterministic (seeded by day-of-year) so all players receive the same 3 quests on any given day, enabling social coordination.

Sample Quest Template Pool (30 Total)

Quest	Difficulty	Coins	Gems	Est. Time
Catch 5 fish	Easy	40	—	10 min
Visit 5 different rooms	Easy	30	—	5 min
Sign 3 guestbooks	Easy	35	—	10 min
Assemble 3 Perfect pizzas	Medium	75	—	25 min
Complete a P2P trade	Medium	50	—	15 min
Spend 30 minutes in the game	Hard	100	—	30 min
Craft a workshop item	Hard	120	1	45 min

New Prisma Schema

```
model DailyQuestProgress {
  id          String  @id @default(cuid())
  userId      String
  questId     String
  progress    Int     @default(0)
  goal        Int
  completed   Boolean @default(false)
  rewardCoins Int
  rewardGems  Int     @default(0)
  questDate   DateTime
  user        User    @relation(fields: [userId], references: [id])
}
```

```
@@unique([userId, questId, questDate]) }
```

Implementation Steps

90. **LINUX** Add `DailyQuestProgress` model. Run: `npm prisma migrate dev --name add_quests`
91. **LINUX** Create `src/shared/quests.ts` — 30-quest template catalog and `getDailyQuests(date: Date)` seeded selection function (deterministic; same result for all players on a given UTC day).
92. **LINUX** Create `src/server/quests.ts` — `QuestService`: `initDailyQuests(userId)`, `incrementProgress(userId, action)`, `completeQuest(userId, questId)` with coin/gem award and Socket.io `QUEST_COMPLETE` notification.
93. **LINUX** Call `QuestService.incrementProgress()` from all relevant server handlers: fish caught, pizza completed, room entered, guestbook signed, trade completed, items crafted.
94. **MAC** Build Quest HUD widget: compact panel showing 3 active quests with icons, progress bars, reward badges, and time-to-refresh countdown.

\$16 Photo Booth & Room Screenshots ★ New Addition

File references: `src/server/gallery.ts` `src/client/gallery/GalleryPanel.ts`

Players take in-game screenshots of themselves and their rooms using Babylon.js's native canvas capture, optionally share them to a public Haven Gallery feed, and receive community likes. Storage uses Oracle Cloud Object Storage (always-free 10 GB tier).

Photo System Design

- Screenshot captured client-side:
`engine.getRenderingCanvas().toDataURL("image/png")` — no server render required
- Pre-signed Oracle Cloud Object Storage URL generated server-side; client uploads directly — API server never handles the binary payload
- Moderation: all uploaded images are perceptual-hashed using the `blockhash-core` npm package; hash stored in DB and compared against a blocklist of known inappropriate image hashes before appearing publicly
- Limits: max 5 photo uploads per player per day; max 50 total stored photos per player
- Caption: optional text up to 80 characters
- Gallery feed: sorted by most liked then newest; each card shows avatar name, room name, like count, and timestamp
- Likes: one like per player per photo; server-side uniqueness enforced via `PhotoLike @@unique([photoId, userId])`

New Prisma Schema

```
model GalleryPhoto {
  id          String @id @default(cuid())
  authorId    String
  imageUrl    String
  caption     String? @db.VarChar(80)
  likes       Int @default(0)
  roomName    String
  perceptualHash String?
  createdAt   DateTime @default(now())
  author      User @relation(fields: [authorId], references: [id])
  photoLikes  PhotoLike[]
}
model PhotoLike {
  id      String @id @default(cuid())
  photoId String
  userId  String
  photo   GalleryPhoto @relation(fields: [photoId], references: [id])
  user    User @relation(fields: [userId], references: [id])
  @@unique([photoId, userId])
}
```

Oracle Cloud Object Storage Setup

[LINUX] OCI Object Storage Configuration

- Create bucket in Oracle Cloud Infrastructure Console: region = your VM region, visibility = Private
- Always-free tier: up to 10 GB storage, up to 50,000 API calls/month
- Generate pre-authenticated request URLs server-side using `oci objectstorage generate_preauthenticated_url`: `oci objectstorage generate_preauthenticated_url`
`install oci-sdk`
- Client uploads directly to the pre-signed URL — API server never proxies binary data
- Set bucket lifecycle policy: auto-delete objects older than 365 days to stay within free tier

Implementation Steps

95. **LINUX** Set up Oracle Cloud Object Storage bucket in OCI console. Install OCI SDK: `npm install oci-sdk`. Implement pre-signed URL generator utility.
96. **LINUX** Add `GalleryPhoto` and `PhotoLike` models. Run: `npm run prisma migrate dev --name add_gallery`
97. **MAC** Add screenshot button to HUD: calls `engine.getRenderingCanvas().toDataURL("image/png")`, opens preview modal with caption input and Upload button.
98. **LINUX** Implement `POST /api/gallery` endpoint: validates file size (<2 MB), checks daily upload limit (max 5), generates pre-signed OCI URL, saves `GalleryPhoto` record with perceptual hash.
99. **MAC** Build Gallery panel in the client: infinite scroll feed, like button (heart icon), photo cards showing avatar name, room name, caption, like count, and timestamp.

Appendix A — Migration Consolidation: Full Phase 3 Prisma Plan

Run all migrations in this sequence on the Oracle Cloud Linux server. Each migration is atomic — run them in order without skipping. After all migrations complete, regenerate the Prisma client and reload PM2.

⚠ Pre-Migration Checklist

Before running any migration: take a PostgreSQL snapshot (`pg_dump havenworld > backup_$(date +%Y%m%d).sql`), ensure PM2 process is idle (no active player sessions in production), and verify disk space on the VM has at least 500 MB free.

#	Migration Name	Tables / Changes Added
1	add_fishing	FishCatch, FishingLeaderboard
2	add_room_privacy	RoomPrivacy enum, RoomDecorator, RoomAccessLog; add privacy/passwordHash/awayMessage to Rooms
3	add_guestbook_tipjar	GuestbookEntry, TipTransaction
4	add_economy	DailyLoginStreak, GiftTransaction; add havenGems to Users
5	add_pets	Pet (with PetType, PetState enums)
6	add_minigames	MinigameSession, WeeklyEarningsCap
7	add_workshop	MaterialInventory, CraftingQueue
8	add_passport_achievements	Achievement, PassportFrame; add

#	Migration Name	Tables / Changes Added
		isVIP/vipSince to Users
9	add_room_mood	Add moodPreset to Rooms
10	add_seasonal_events	SeasonalEvent, SeasonalProgress
11	add_clubs	Club, ClubMember (with ClubRole enum)
12	add_quests	DailyQuestProgress
13	add_gallery	GalleryPhoto, PhotoLike

After all migrations complete:

[LINUX] Final Post-Migration Commands

```
npx prisma generate pm2 reload havenworld pm2 logs havenworld --lines 50 #
Verify no startup errors
```

Appendix B — Phase 3 Deployment Checklist

[LINUX] Install New npm Packages

```
npm install bad-words node-cron @babylonjs/recast npm install oci-sdk npm install
blockhash-core
```

[MAC] TypeScript Type Updates

Update `src/shared/types.ts` to include all new Socket.io event payloads added in Phase 3: all fish events, doorbell events, trade events, economy events, pet events, quest events, and gallery events.

[MAC] Client Build & Deploy

```
npm run build git add -A && git commit -m "feat: Phase 3 complete" git push origin main # Triggers Cloudflare Pages CI/CD via GitHub Actions
```

[LINUX] Verify Cron Jobs After PM2 Reload

- **Fishing leaderboard reset:** Every Monday 00:00 UTC
- **Pet stat decay (happiness & hunger):** Every hour
- **Crafting queue completion check:** Every 5 minutes
- **Flash sale scheduler:** 2-3× per day (random time via node-cron)
- **Seasonal currency conversion:** On event end date (one-time cron)

[LINUX] Nginx Configuration Note

No Nginx config changes are required for Phase 3 if all new endpoints follow the `/api/` path prefix already proxied in the Phase 0 configuration. New routes: `/api/passport/user/`, `/api/gallery/`, `/api/admin/economy-stat/` — all are covered by the existing wildcard proxy rule.

Appendix C — Implementation Sprint Order & Feature Summary

Recommended Phase 3 Sprint Plan

Sprint	Features	Estimated Effort	Engagement Impact
Sprint 3.1	Dual Currency Economy, Daily Quests, Login Streak	2 weeks	Immediate retention baseline
Sprint 3.2	Fishing Mini-Game, Pizza Chef, Pet Companions	3 weeks	Core activity loops
Sprint 3.3	Loft Privacy/Doorbell, Guestbook/Tip Jar, Ambient Moods	2 weeks	Social loft culture
Sprint 3.4	Workshop Crafting, P2P Trading, Player Passport	3 weeks	Depth & economy health
Sprint 3.5	Seasonal Events, Club System, Emote Wheel	3 weeks	Community & replayability
Sprint 3.6	Procedural Sound Engine, Photo Gallery	2 weeks	Polish & sharing

Total estimated Phase 3 timeline: 15 weeks (solo developer pace, including testing time)

Complete Phase 3 Feature Summary

Feature	Priority	DB Tables Added	Socket Events	Environment
Fishing Mini-Game	High	FishCatch, FishingLeader board	8	MAC + LINUX

Feature	Priority	DB Tables Added	Socket Events	Environment
Loft Privacy/Doorbell	High	RoomDecorator, RoomAccessLog	7	MAC + LINUX
Guestbook/Tip Jar	Medium	GuestbookEntry, TipTransaction	6	MAC + LINUX
Procedural Sound Engine	Medium	None	0 server events	MAC only
P2P Trading	High	TradeLog	10	MAC + LINUX
Dual Currency/Shop	High	DailyLoginStreak, GiftTransaction	8	MAC + LINUX
Pet Companions	High	Pet	6	MAC + LINUX
Pizza Chef	Medium	MinigameSession, WeeklyEarningsCap	3	MAC + LINUX
Workshop Crafting	Medium	MaterialInventory, CraftingQueue	5	MAC + LINUX
Player Passport	High	Achievement, PassportFrame	2	MAC + LINUX
Ambient Moods	Low	moodPreset field on Rooms	2	MAC + LINUX
Seasonal Events	High	SeasonalEvent, SeasonalProgress	4	MAC + LINUX
Club System	High	Club, ClubMember	5	MAC + LINUX
Emote Wheel	Medium	None	2	MAC + LINUX
Daily Quests	High	DailyQuestProgress	3	MAC + LINUX

Feature	Priority	DB Tables Added	Socket Events	Environment
Photo Gallery	Medium	GalleryPhoto, PhotoLike	2	MAC + LINUX

★ Phase 3 Closing Notes — Ready for Beta

Phase 3 completes HavenWorld's transformation from a functional multiplayer tech demo into a rich, feature-complete social virtual world ready for a broader beta launch. Every feature above is implementable entirely within the free-tier infrastructure established in Phases 0-2:

- **Oracle Cloud Free Tier ARM VM** — handles all server-side game logic, Socket.io real-time events, Prisma/PostgreSQL, Redis pub/sub, and all node-cron scheduled jobs
- **Cloudflare Pages** — serves all Babylon.js client assets at zero cost with global CDN edge caching
- **Oracle Cloud Object Storage** — always-free 10 GB handles all Photo Gallery uploads via pre-signed direct client upload
- **nip.io** — provides stable DNS for the API endpoint without a custom domain purchase
- **No paid services** — zero cost introduced at any point across all of Phase 3

After Phase 3 is Complete

HavenWorld will have: 2 economy currencies with healthy sinks and sources — 2 mini-games with leaderboards — a full crafting and recycling workshop — adoptable AI-driven pet companions — a social trading system with anti-scam

protections — player passports with 20 achievement stamps — 8 ambient room moods — a seasonal content calendar — player clubs with clubhouses — 12 expressive emotes — daily quests — a community photo gallery — and a procedural zero-asset sound system. This is a complete, launch-ready social virtual world built entirely on free infrastructure.

HavenWorld Build Guide Part 6 — Authored by Daniel — Anoka, MN — Friday, 18 September 2026, 15:52 CDT

This document is Part 6 of an ongoing series. All content describes original game systems built from scratch. No content from existing commercial games is reproduced herein.

Next: Part 7 will cover Phase 4 — Beta Launch Preparations, Analytics, and Anti-Abuse Infrastructure.